Public, Private and Class Attibutes, Class and Static Methods

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY – INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

What is a Public Attribute?

A **public attribute** is **accessible from anywhere**: inside the class, in subclasses, and from outside the class. **No underscore prefix** is used.

Key Points

- Public attributes are **fully accessible and modifiable**.
- Use when you want to expose behavior or data **without restrictions**.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Public Attribute – Basic Example

```
class Animal:
  def __init__(self) -> None:
    self.name:str = "Generic Animal" # Public attribute
```

```
animal:Animal = Animal()
print(animal.name) # Output: "Generic Animal"


animal.name = "It's a dog" # MODIFIES the public attribute 'name'
print(animal.name) # Output: "It's a dog"
```

They work.. but are extremely dangerous!

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Risks of Public Attributes

Here's why:

- **No protection**: Any external code can **change** the value at any time.
- **No control**: You can't enforce rules like type-checking, validation, or logging.
- **Can lead to bugs**: Unintended changes might cause inconsistent or invalid states.

Alternative:

A safer approach would involve:

- Using **private attributes** (prefix with _ or __).
- Accessing via **getter/setter methods** to maintain control.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

What is a Private Attribute?

A **private attribute** is intended to be **completely hidden** from outside the class. Defined using **double underscore** prefix: __attribute.

Key Points

- Python performs **name mangling**: renames __attribute internally as _ClassName__attribute.
- Not truly “private”, but **discourages** external access.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Private Attribute – Basic Example/1

```
class Animal:
  def __init__(self) -> None:
    self.__name:str = "Generic Animal" # Private attribute
```

```
animal:Animal = Animal()
print(animal.name) # ✖ Error: not accessible directly

animal.name = "It's a dog" # Creates a NEW public attribute 'name'
print(animal.name) # Output: "It's a dog"
```

They no longer work!

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Private Attribute – Basic Example/2

```
class Animal:
  def __init__(self) -> None:
    self.__name:str = "Generic Animal" # Private attribute
```

```
  # ✅ Makes 'name' accessible again
  def get_name(self) -> str:
    return self.__name


animal:Animal = Animal()
print(animal.get_name()) # Output: "Generic Animal"
```

They work again!

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Static Methods and Class Attributes

- **Static Method**: defined with `@staticmethod`, does not access instance or class.

- **Class Attribute**: shared by all instances of the class.

Useful for common behavior and shared constants.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Class Attributes – Basic Example

```
class Person:
  population:int = 0 # Public class attribute

  def __init__(self, name:str) -> None:
    self.__name:str = name # Private attribute
    Person.population += 1


print(Person.population) # Output: 0
person1:Person = Person("Alice")
print(Person.population) # Output: 1
person2:Person = Person("Bob")
print(Person.population) # Output: 2
```

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Static Methods – Basic Example

```
class Math:
  PI:float = 3.14 # Public class attribute

  @staticmethod
  def circle_area(radius:float) -> float: # Static Method
    return Math.PI * radius * radius


print(Math.PI) # Output: 3.14
print(Math.circle_area(5)) # Output: 78.5
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

What is a Class Method?

A **class method** is a method that is **bound to the class**, not the instance.

Key concepts

- It is defined with `@classmethod`
- It takes **cls** (not self) as its **first parameter**.
- It can **access and modify class-level attributes**.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Class Methods – Basic Example

```
class Person:
  __population:int = 0 # Private class attribute
  def __init__(self, name:str) -> None:
    self.__name:str = name # Private attribute
    Person.__population += 1

  @classmethod
  def get_population(cls) -> None:
    return cls.__population

print(Person.get_population()) # Output: 0
person1:Person = Person("Alice")
print(Person.get_population()) # Output: 1
person2:Person = Person("Bob")
print(Person.get_population()) # Output: 2
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY